

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторных работ
«Программирование и настройка СУБД»
по дисциплине «Методы проектирования баз данных»

Студент
группы 5130201/30101

Мартыненко А. П.

Преподаватель

Попов С.Г.

№	Название работы	Дата сдачи
1	Представления	24.09.2025
2	Триггеры	01.10.2025
3	Права пользователей	08.10.2025
4	Функции и процедуры	22.10.2025
5	Транзакции	01.10.2025

02.12 2025г.

Санкт-Петербург, 2025

Реферат

Реляционные СУБД позволяют создавать базы данных, определять структуру данных, взаимодействовать с данными через запросы и обеспечивать защиту информации. В данной работе реализованы такие элементы, как представления, триггеры, процедуры, функции, транзакции и система прав пользователей. Эти компоненты обеспечивают безопасность, согласованность и целостность данных, разграничение доступа и атомарность операций. Все элементы были написаны на языке SQL, для выполнения запросов и создания объектов базы данных использовалась среда MySQL Workbench 8.0 CE, работа с пользователями выполнялась с помощью командной строки. Разработанная система может применяться в коммерческих бассейнах для автоматизации учёта посещений и финансов, а также возможности многопользовательского взаимодействия.

Содержание

Введение	4
1 Постановка задачи	5
2 Основная часть	6
2.1 Представления	6
2.2 Триггеры	9
2.3 Права пользователей	16
2.4 Функции и процедуры	20
2.5 Транзакции	24
Заключение	27
Список использованных источников	28
Приложение А «Исходный текст функции»	29
Приложение Б «Исходный текст процедуры»	30

Введение

Работа с большими объемами данных требует системного подхода к их организации, хранению и быстрому доступу. При ручной обработке данных возрастает риск ошибок, утерь информации, а также значительно увеличивается время на поиск и анализ необходимых сведений.

Система баз данных - это компьютеризированная система хранения однотипных записей. Саму базу данных можно рассматривать как подобие электронной картотеки, то есть хранилище или контейнер для некоторого набора данных, занесенных в компьютер. Она хранит информацию структурированно и упрощает её обработку, обеспечивая целостность, согласованность и защиту данных. [1]

Ключевыми преимуществами современных СУБД являются обеспечение безопасности доступа к данным, поддержание целостности информации и согласованности данных при параллельной работе множества пользователей. Система обеспечивает разграничение доступа на уровне отдельных операций для разных категорий пользователей и поддерживает атомарность операций через механизм транзакций, когда группа действий выполняется либо полностью, либо не выполняется вовсе. [3]

Внедрение баз данных позволяет ускорить выполнение рутинных операций, минимизировать дублирование и обеспечить одновременный доступ нескольких пользователей к массиву данных. Это может быть полезно для эффективного управления бассейном и планирования бюджета организации.

Взаимодействие с базами данных, а именно редактирование, добавление и удаление данных, написание и выполнение запросов осуществляется с помощью языка SQL. Этот язык позволяет получать выборки по заданным критериям для детального анализа, а также обеспечивает быстрый доступ к информации даже при ее постоянном обновлении и увеличении объема. [2]

1 Постановка задачи

Задачи.

1. Создать представление на основе выбранного агрегирующего запроса и написать запрос к этому представлению.
2. Определить и написать триггеры, автоматизирующие изменения статистики.
3. Создать двух пользователей с разными правами доступа и проверить их ограничения.
4. Создать функцию и процедуру для заданной базы данных.
5. Показать наличие или отсутствие выбранной транзакционной аномалии на заданном уровне изоляции.

2 Основная часть

2.1 Представления

Постановка задачи.

1. Создать представления на основе выполненного ранее запроса с агрегатными функциями.
2. Выполнить простого запроса к этому представлению.
3. Выполнить сложного запроса с присоединением других таблиц и представления.
4. Проверить обновляемость представлений.

Запрос, на основе которого было построено представление: посчитать количество посещений, которые зарегистрировал каждый администратор. Код создания view представлен в листинге 1.

Листинг 1. Создания представления view_admin

```
1 CREATE VIEW view_admin as
2 SELECT
3 admin.admin_surname as adm_surname,
4 admin.admin_name as adm_name,
5 admin.admin_patronymic as adm_patr,
6 COUNT(visit.id_visit) as count_visit
7 FROM Admin
8 INNER JOIN Visit ON admin.id_admin = visit.id_admin
9 GROUP BY admin.admin_surname, admin.admin_name, admin.admin_patronymic;
```

Далее был составлен простой запрос, который выводит только администраторов, зарегистрировавших более 4900 посещений. Код примера использования представления в простом запросе представлен в листинге 2.

Листинг 2. Простой запрос к view_admin

```
1 SELECT
2 adm_surname as "фамилия",
3 adm_name as "имя",
4 adm_patr as "отчество",
5 count_visit as "посещения"
6 FROM view_admin
7 WHERE count_visit > 4990
8 ORDER BY count_visit;
```

фамилия	имя	отчество	посещения
Голиков	Каролина	Семёновна	4996
Щеглов	Олеся	Климовна	4999
Щербаков	Инна	Ратмирович	5001
Уткин	Оскар	Илларионовна	5007
Щекотихин	Еремей	Добрыняовна	5009
Рублев	Всеволод	Алексейович	5012
Заец	Любомир	Русланович	5013
Щекотихин	Валерий	Добрыняович	5015
Борисов	Август	Геннадийовна	5017
Архипов	Герман	Всеволодовна	5021

Рис. 1. Результат простого запроса

Сложный запрос заключался в выводе только тех администраторов, которые впервые зарегистрировали посетителей позже 06.01.2012. Для выполнения данного запроса необходимо было с помощью inner join соединить представление и таблицы Admin, Purchase. Код примера использования представления в сложном запросе представлен в листинге 3.

Листинг 3. Сложный запрос к view_admin

```

1  SELECT
2  adv.adm_surname as "фамилия",
3  adv.adm_name as "имя",
4  adv.adm_patr as "отчество",
5  adv.count_visit as "посещения",
6  MIN(p.purchase_date) as "дата"
7  FROM view_admin as adv
8  INNER JOIN Admin as a on
9  adv.adm_surname = a.admin_surname and
10 adv.adm_name = a.admin_name and
11 adv.adm_patr = a.admin_patronymic
12 INNER JOIN Purchase as p on a.id_admin=p.id_admin
13 WHERE a.id_admin IN (
14 SELECT p2.id_admin
15 FROM Purchase p2
16 GROUP BY p2.id_admin
17 HAVING MIN(p2.purchase_date) > '2012-01-06'
18 )
19 GROUP BY adv.adm_surname, adv.adm_name, adv.adm_patr, adv.count_visit
20 ORDER BY MIN(p.purchase_date);

```

фамилия	имя	отчество	посещения	дата
Андрианов	Валентина	Оскарovich	5075	2012-01-07
Щеглов	Лилия	Павеловна	5034	2012-01-07
Кузнецов	Снежана	Григорийовна	5068	2012-01-08
Заец	Ксения	Брониславовна	5053	2012-01-08
Титов	Богдан	Мирославович	5031	2012-01-09
Медведев	Клавдия	Аркадийовна	4930	2012-01-10
Панин	Василина	Романовна	5082	2012-01-10
Сухарев	София	Олегович	5061	2012-01-11
Щекотихин	Еремей	Добрыняовна	5009	2012-01-12
Рублев	Всеволод	Алексейович	5012	2012-01-13

Рис. 2. Результат сложного запроса

Представление называется обновляемым, если к нему могут быть применимы операторы для изменения данных в таблицах, на которых основано представление. Для того, чтобы представление было обновляемым, должно быть выполнено два условия:

1. Каждой строке представления должно соответствовать по одной строке в таблицах-источниках (отношение 1 к 1).
2. Поля представления должны быть простым перечислением полей таблиц.

Представление, основанное на нескольких таблицах, может допускать обновление при объединении его таблиц с помощью inner join.

В данном представлении есть агрегатная функция count, которая объединяет множество строк из таблицы Visit в одну строку для результата каждого администратора. Также используется group by, которое группирует данные и не гарантирует отношения 1 к 1. Таким образом, данное представление является не обновляемым. Результат попыток изменения приведен на рисунке 3. Код проверки обновляемости представления приведен в листинге 4.

Листинг 4. Проверка обновляемости view_admin

```

1 DELETE FROM view_admin
2 WHERE view_admin.surname = "Андрианов";
3
4 INSERT INTO view_admin (adm_surname, adm_name, adm_pat, count_visit)
5 VALUES ('Иванов', 'Иван', 'Иванович', 3000);

```

Error Code: 1288. The target table view_admin of the DELETE is not updatable

Error Code: 1471. The target table view_admin of the INSERT is not insertable-into

Рис. 3. Проверка обновляемости

2.2 Триггеры

Постановка задачи.

1. Создать новую таблицу с для подсчета статистики с тремя полями: id, название и количество.
2. Изменить схему базы данных в соответствии с выполненными изменениями.
3. Определить и написать триггеры на добавление, обновление и удаление, чтобы статистика считалась корректно.
4. Проверить корректность работы триггеров.

Необходимо написать триггер, который меняет количество абонементов конкретного типа в зависимости от добавления, изменения и удаления абонементов. Также нужно учесть ситуацию изменения данных о пользователях, купивших абонементы.

Для выполнения задачи была создана дополнительная таблица для подсчета количества типов абонементов, состоящая из трех столбцов: id типа абонемента, его название и количество купленных абонементов данного типа. С помощью запросов к исходным столбцам таблица была заполнена. Код создания таблицы представлен в листинге 5.

Обновленная схема базы данных на английском языке с именами триггеров и направлением их действия представлена на рисунке 4.

Листинг 5. Создание таблицы pass_type_stat

```
1 create table if not exists pass_type_stat(  
2 id_type_of_pass int not null,  
3 type_name varchar(20) not null,  
4 count_type int not null);  
5  
6 insert into pass_type_stat(id_type_of_pass, type_name, count_type)  
7 select 'Type_of_pass'.id_type_of_pass,  
8 'Type_of_pass'.type_name,  
9 count('Pass'.id_pass)  
10 from 'Pass' inner join 'Type_of_pass' on 'Type_of_pass'.id_type_of_pass  
    = 'Pass'.id_type_of_pass  
11 group by 'Pass'.id_type_of_pass;
```

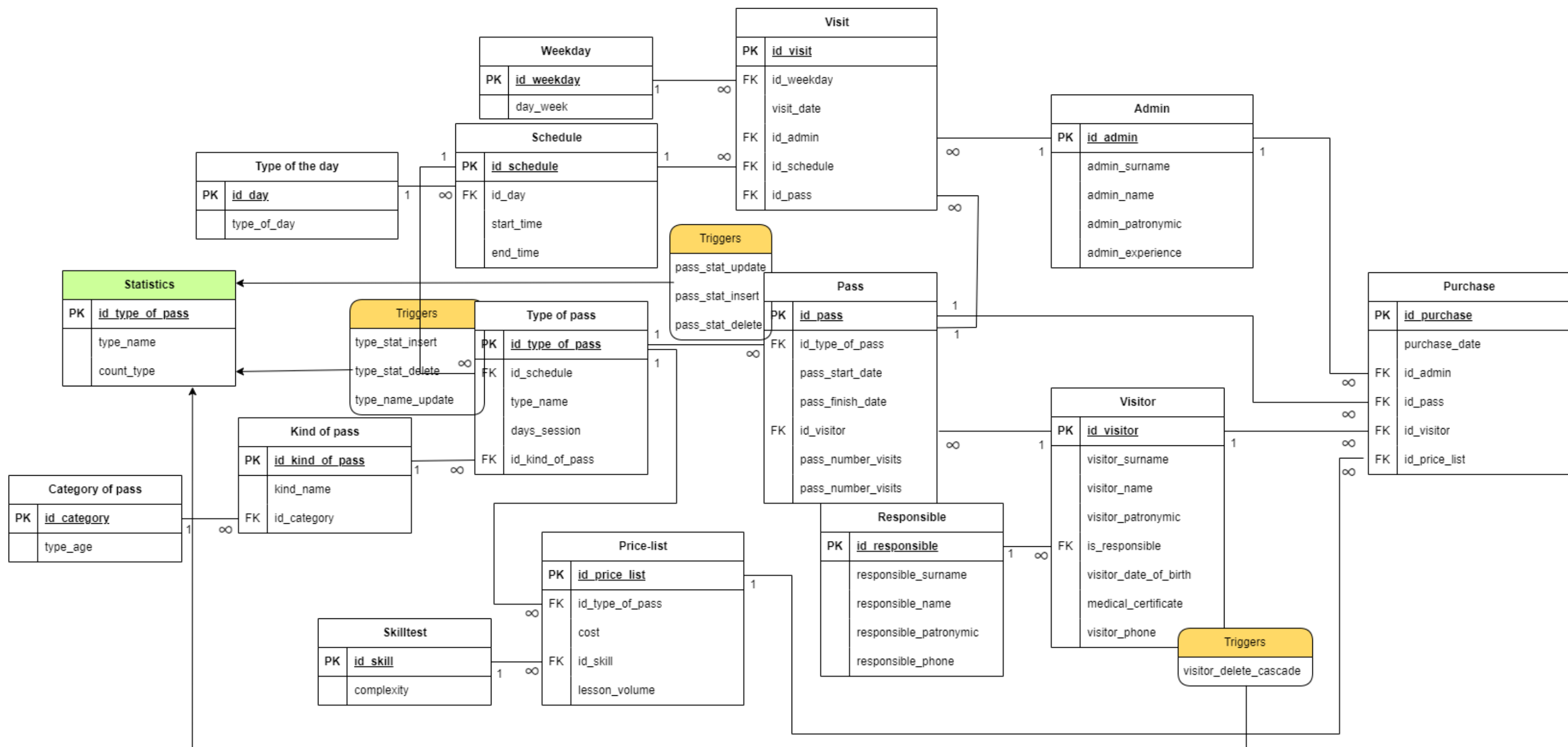


Рис. 4: Графическое описание схемы базы данных на английском языке

Пример вывода новой таблицы представлен на рисунке 5.

	id_type_of_pass	type_name	count_type
►	1	Стандарт	469
	2	Базовый	518
	3	Оптимальный	461
	4	Комфорт	536
	5	Универсальный	492
	6	Профи	500

Рис. 5. Новая таблица со статистикой

Для обработки изменений в таблице Type_of_pass были написаны три триггера: на добавление, обновление и удаление. При добавлении нового типа абонента необходимо установить количество этих абонентов равное нулю, при удалении удалить необходимую строку таблицы. При обновлении имя типа меняется на новое. Код создания триггеров представлен в листинге 6.

Листинг 6. Триггеры для таблицы Type_of_pass

```
1 create trigger type_stat_insert after insert on 'Type_of_pass'
2 for each row
3 insert into pass_type_stat (id_type_of_pass, type_name, count_type)
4 values (NEW.id_type_of_pass, NEW.type_name, 0);
5
6 create trigger type_stat_delete after delete on 'Type_of_pass'
7 for each row
8 delete from pass_type_stat
9 where id_type_of_pass = OLD.id_type_of_pass;
10
11 create trigger type_name_update after update on 'Type_of_pass'
12 for each row
13 update pass_type_stat
14 set type_name = NEW.type_name
15 where id_type_of_pass = NEW.id_type_of_pass;
```

Таблица 1. Перечень тестовых примеров для триггеров таблицы Type_of_pass

№	Описание проверяемого состояния																								
1	При добавлении нового типа «Новая волна» количество абонементов данного типа становится равно нулю. <table><tr><th></th><th>id_type_of_pass</th><th>type_name</th><th>count_type</th></tr><tr><td></td><td>197</td><td>Акваджет</td><td>513</td></tr><tr><td></td><td>198</td><td>Аквастрим</td><td>495</td></tr><tr><td></td><td>199</td><td>Акваклуб</td><td>510</td></tr><tr><td></td><td>200</td><td>Аквалидер</td><td>461</td></tr><tr><td></td><td>205</td><td>Новая волна</td><td>0</td></tr></table>		id_type_of_pass	type_name	count_type		197	Акваджет	513		198	Аквастрим	495		199	Акваклуб	510		200	Аквалидер	461		205	Новая волна	0
	id_type_of_pass	type_name	count_type																						
	197	Акваджет	513																						
	198	Аквастрим	495																						
	199	Акваклуб	510																						
	200	Аквалидер	461																						
	205	Новая волна	0																						
2	При изменении типа данного элемента с «Новая волна» на «Супер волна» меняется название соответствующего типа абонемента. <table><tr><th></th><th>id_type_of_pass</th><th>type_name</th><th>count_type</th></tr><tr><td></td><td>197</td><td>Акваджет</td><td>513</td></tr><tr><td></td><td>198</td><td>Аквастрим</td><td>495</td></tr><tr><td></td><td>199</td><td>Акваклуб</td><td>510</td></tr><tr><td></td><td>200</td><td>Аквалидер</td><td>461</td></tr><tr><td></td><td>205</td><td>Супер волна</td><td>0</td></tr></table>		id_type_of_pass	type_name	count_type		197	Акваджет	513		198	Аквастрим	495		199	Акваклуб	510		200	Аквалидер	461		205	Супер волна	0
	id_type_of_pass	type_name	count_type																						
	197	Акваджет	513																						
	198	Аквастрим	495																						
	199	Акваклуб	510																						
	200	Аквалидер	461																						
	205	Супер волна	0																						
3	При удалении абонемента типа «Супер волна» строка с количеством абонементов данного типа удаляется. <table><tr><th></th><th>id_type_of_pass</th><th>type_name</th><th>count_type</th></tr><tr><td></td><td>196</td><td>Аквафлоу</td><td>461</td></tr><tr><td></td><td>197</td><td>Акваджет</td><td>513</td></tr><tr><td></td><td>198</td><td>Аквастрим</td><td>495</td></tr><tr><td></td><td>199</td><td>Акваклуб</td><td>510</td></tr><tr><td></td><td>200</td><td>Аквалидер</td><td>461</td></tr></table> pass_type_stat9 ×		id_type_of_pass	type_name	count_type		196	Аквафлоу	461		197	Акваджет	513		198	Аквастрим	495		199	Акваклуб	510		200	Аквалидер	461
	id_type_of_pass	type_name	count_type																						
	196	Аквафлоу	461																						
	197	Акваджет	513																						
	198	Аквастрим	495																						
	199	Акваклуб	510																						
	200	Аквалидер	461																						

Для обработки изменений в таблице Pass были написаны три триггера: на добавление, обновление и удаление. При добавлении абонемента конкретного типа необходимо увеличить количество этих абонементов на единицу, при удалении - уменьшить на единицу. При обновлении количество «старого» типа уменьшается на единицу, а «нового» - увеличивается. Код представлен в листинге 7.

Листинг 7. Триггеры для таблицы Pass

```

1 create trigger pass_stat_insert after insert on 'Pass'
2 for each row
3 update pass_type_stat
4 set count_type = count_type+1
5 where id_type_of_pass = NEW.id_type_of_pass;
6
7 create trigger pass_stat_delete after delete on 'Pass'
8 for each row
9 update pass_type_stat
10 set count_type = count_type-1
11 where id_type_of_pass = OLD.id_type_of_pass;
12
13 delimiter |
14 create trigger pass_stat_update after update on 'Pass'
15 for each row
16 begin
17 update pass_type_stat
18 set count_type = count_type-1
19 where id_type_of_pass = OLD.id_type_of_pass;
20 update pass_type_stat
21 set count_type = count_type+1
22 where id_type_of_pass = NEW.id_type_of_pass;
23 end |
24 delimiter ;
25

```

Таблица 2. Перечень тестовых примеров для триггеров таблицы Pass

№	Описание проверяемого состояния		
1	При добавлении одного нового элемента типа «Стандарт» количество абонементов первого типа увеличивается.		
		id_type_of_pass	type_name
		count_type	
	▶	1	Стандарт
		2	Базовый
		3	Оптимальный
		4	Комфорт
		5	Универсальный
		6	Профи

№	Описание проверяемого состояния		
2	При изменении типа данного элемента со «Стандарт» на «Базовый» меняются две колонки количества абонементов.		
		id_type_of_pass	type_name
	▶ 1	Стандарт	469
	2	Базовый	519
	3	Оптимальный	461
	4	Комфорт	536
	5	Универсальный	492
	6	Профи	500
3	При удалении абонементов типа «Базовый» количество абонементов второго типа уменьшается.		
		id_type_of_pass	type_name
	▶ 1	Стандарт	469
	2	Базовый	518
	3	Оптимальный	461
	4	Комфорт	536
	5	Универсальный	492
	6	Профи	500

Для обработки изменений в таблице Visitor был создан триггер на удаление. Перед удалением посетителя удаляются все его абонементы, соответственно их количество тоже уменьшается. Код представлен в листинге 8.

Листинг 8. Триггеры для таблицы Visitor

```

1 create trigger visitor_delete_cascade before delete on 'Visitor'
2 for each row
3 delete from 'Pass' where id_visitor = old.id_visitor;
```

При добавлении одного нового посетителя добавим ему два абонемента второго типа. В результате абонементов типа «Базовый» будет 520. При удалении посетителя автоматически удаляются все абонементы, связанные с ним. Результат приведен на рисунке 6.

	id_type_of_pass	type_name	count_type
▶	1	Стандарт	469
	2	Базовый	518
	3	Оптимальный	461
	4	Комфорт	536
	5	Универсальный	492
	6	Профи	500

Рис. 6. Проверка триггера delete

2.3 Права пользователей

Постановка задачи.

1. Создать двух пользователей с различными правами доступа.
2. Проверить работоспособность операций insert, delete, update, show для каждого из пользователей.

Для выполнения задания было создано два пользователя: user3 и superuser. Пользователь user3 может только просматривать таблицу pass_type_stat. Пользователь superuser может редактировать и просматривать данные таблиц, которые связаны с таблицей pass_type_stat, а также просматривать саму таблицу pass_type_stat. Код создания пользователей приведен в листинге 9. Информация о правах пользователей представлена на рисунках 7 - 8.

Листинг 9. Создание пользователей с разными правами доступа

```
1 create user 'user3'@'localhost' identified by 'pass3';
2 create user 'superuser'@'localhost' identified by 'superpass';
3
4 grant select on swimming_pool.pass_type_stat to 'user3'@'localhost';
5 grant select on swimming_pool.pass_type_stat to 'superuser'@'localhost';
6 grant select, update, delete, insert on swimming_pool.type_of_pass to '
  superuser'@'localhost';
7 grant select, update, delete, insert on swimming_pool.visitor to '
  superuser'@'localhost';
8 grant select, update, delete, insert on swimming_pool.pass to 'superuser
  '@'localhost';
9 flush privileges;
```

	User	Database	Table	Privilege
►	'user3'@'localhost'	swimming_pool	view1	SELECT

Рис. 7. Список прав для user3

	User	Database	Table	Privilege
►	'superuser'@'localhost'	swimming_pool	pass	SELECT
	'superuser'@'localhost'	swimming_pool	pass	INSERT
	'superuser'@'localhost'	swimming_pool	pass	UPDATE
	'superuser'@'localhost'	swimming_pool	pass	DELETE
	'superuser'@'localhost'	swimming_pool	type_of_pass	SELECT
	'superuser'@'localhost'	swimming_pool	type_of_pass	INSERT
	'superuser'@'localhost'	swimming_pool	type_of_pass	UPDATE
	'superuser'@'localhost'	swimming_pool	type_of_pass	DELETE
	'superuser'@'localhost'	swimming_pool	view1	SELECT
	'superuser'@'localhost'	swimming_pool	visitor	SELECT

Рис. 8. Список прав для superuser

Для проверки корректности работы прав пользователей был выполнен ряд команд с каждой из используемых таблиц (pass_type_stat, type_pass, pass, visitor). Результаты их выполнения для разных пользователей показаны в таблице 3.

Из таблицы видно, что user3 может только просматривать view с таблицей pass_type_stat, попытки взаимодействия с другими таблицами для него невозможно. Пользователь superuser может изменять все таблицы, кроме view. Соответственно, оба пользователя действуют в рамках своих прав доступа.

Таблица 3. Проверка прав доступа пользователей

№	User3	Superuser
1	Просмотр содержимого view1: <code>select * from view1;</code>	
	<pre> id_type_of_pass type_name count_type ----- 1 Стандарт 469 2 Базовый 518 3 Оптимальный 461 4 Комфорт 536 5 Универсальный 492 </pre> Успех	<pre> id_type_of_pass type_name count_type ----- 1 Стандарт 469 2 Базовый 518 3 Оптимальный 461 4 Комфорт 536 5 Универсальный 492 </pre> Успех
2	Удаление view1: <code>drop view view1;</code>	
	<pre> ERROR 1142 (42000): DROP command denied to user 'user3'@'localhost' for table 'view1' </pre> Ошибка	<pre> ERROR 1142 (42000): DROP command denied to user 'superuser'@'localhost' for table 'view1' </pre> Ошибка
3	Добавление данных во view1: <code>insert into view1 (id_type_of_pass, type_name, count_type) values (55, 'Акула', 12);</code>	
	<pre> ERROR 1142 (42000): INSERT command denied to use r 'user3'@'localhost' for table 'view1' </pre> Ошибка	<pre> ERROR 1142 (42000): INSERT command denied to use r 'superuser'@'localhost' for table 'view1' </pre> Ошибка
4	Просмотр таблицы Type_of_pass: <code>select * from type_of_pass;</code>	
	<pre> ERROR 1142 (42000): SELECT command denied to use r 'user3'@'localhost' for table 'type_of_pass' </pre> Ошибка	<pre> ***** 1. row ***** id_type_of_pass: 1 id_schedule: 7 days_session: четверг, вторник, понедельник id_kind_of_pass: 3 type_name: Стандарт </pre> Успех
5	Добавление в Type_of_pass: <code>insert into Type_of_pass (type_name, id_schedule, days_session, id_kind_of_pass) values ('Новая волна', 7, 'среда', 5);</code>	
	<pre> ERROR 1142 (42000): INSERT command denied to use r 'user3'@'localhost' for table 'type_of_pass' </pre> Ошибка	<pre> Query OK, 1 row affected (0.01 sec) </pre> Успех

Проверка прав доступа пользователей (продолжение)

№	User3	Superuser
6	Обновление Type_of_pass: update Type_of_pass set type_name = 'Супер волна' where id_type_of_pass = @type_pass_id;	
	ERROR 1142 (42000): UPDATE command denied to user 'user3'@'localhost' for table 'type_of_pass'	Query OK, 1 row affected (0.02 sec) Rows matched: 1 Changed: 1 Warnings: 0
	Ошибка	Успех
7	Удаление из Type_of_pass: delete from Type_of_pass where type_name = 'Супер волна'	
	ERROR 1142 (42000): DELETE command denied to user 'user3'@'localhost' for table 'type_of_pass'	Query OK, 1 row affected (0.01 sec)
	Ошибка	Успех
8	Просмотр таблицы Pass: select * from Pass;	
	ERROR 1142 (42000): SELECT command denied to user 'user3'@'localhost' for table 'pass'	***** 1. row ***** id_pass: 1 id_type_of_pass: 108 pass_start_date: 2021-02-15 pass_finish_date: 2021-08-14 id_visitor: 7021 pass_number_visits: 2
	Ошибка	Успех
9	Добавление в Pass: insert into Pass ('id_type_of_pass', 'pass_start_date', 'pass_finish_date', 'id_visitor', 'pass_number_visits') values (1, '2023-04-28', '2023-10-25', 9952, 8);	
	ERROR 1142 (42000): INSERT command denied to user 'user3'@'localhost' for table 'pass'	Query OK, 1 row affected (0.01 sec)
	Ошибка	Успех
10	Обновление Pass: update Pass set 'id_type_of_pass' = 2 where 'id_pass' = @pass_id;	
	ERROR 1142 (42000): UPDATE command denied to user 'user3'@'localhost' for table 'pass'	Query OK, 1 row affected (0.01 sec) Rows matched: 1 Changed: 1 Warnings: 0
	Ошибка	Успех
11	Удаление из Pass: delete from Pass where 'id_pass' = @pass_id;	
	ERROR 1142 (42000): DELETE command denied to user 'user3'@'localhost' for table 'pass'	Query OK, 1 row affected (0.04 sec)
	Ошибка	Успех

Проверка прав доступа пользователей (продолжение)

№	User3	Superuser
12	Просмотр таблицы Visitor: <code>select * from Visitor;</code>	
	ERROR 1142 (42000): SELECT command denied to use r 'user3'@'localhost' for table 'visitor' Ошибка	<pre> ***** 1. ROW ***** id_visitor: 1 visitor_surname: Медведев visitor_name: Александр visitor_patronymic: Вениаминович visitor_date_of_birth: 2018-07-13 id_responsible: 3105 medical_certificate: 0 visitor_phone: 89138797989 </pre> Успех
13	Добавление Visitor: <code>insert into Visitor (visitor_surname, visitor_name, visitor_date_of_birth, visitor_phone) values ('Иванов', 'Сергей', '1990-05-15', '+79161234567');</code>	
	ERROR 1142 (42000): INSERT command denied to use r 'user3'@'localhost' for table 'visitor' Ошибка	Query OK, 1 row affected (0.00 sec) Успех
14	Обновление Visitor: <code>update Visitor set visitor_name = 'Варварварвара' where id_visitor = @id_visitor;</code>	
	ERROR 1142 (42000): UPDATE command denied to use r 'user3'@'localhost' for table 'visitor' Ошибка	<pre> Query OK, 1 row affected (0.01 sec) Rows matched: 1 Changed: 1 Warnings: 0 </pre> Успех
15	Удаление Visitor: <code>delete from Visitor where id_visitor = @id_visitor;</code>	
	ERROR 1142 (42000): DELETE command denied to use r 'user3'@'localhost' for table 'visitor' Ошибка	Query OK, 1 row affected (0.10 sec) Успех

2.4 Функции и процедуры

Постановка задачи.

1. Создать функцию, возвращающую фамилию и инициалы посетителя по его фамилии, имени и отчеству.
2. Создать процедуру, которая принимает возрастной тип и название вида абонеента, при вызове связывает возрастной тип с данным видом абонеента.

2.4.1 Функция

Для выполнения задания была создана функция `show_initials`, которая принимает три строки типа `varchar`: фамилию, имя и отчество. Функция возвращает сконкатенированную строку в формате «Фамилия И.О.». Если отчество отсутствует, то при составлении инициалов оно пропускается и строка формируется как «Фамилия И.». Код функции представлен в листинге 19.

Результат работы функции приведен на рисунке 9. Представлено корректное преобразование полных инициалов в сокращенные. В случае, если отчество пропущено - сокращается только имя до первой буквы.

Баранов	Иванна	Святославовна	Баранов И.С.
Дубровин	Альфред	Матвейовна	Дубровин А.М.
Сидоров	Анатолий	Васильевич	Сидоров А.В.
Ширяев	Даниил	NULL	Ширяев Д.
Казаков	Нина	Архиповна	Казаков Н.А.
Фадеев	Жаннетта	NULL	Фадеев Ж.
Худякин	Сергей	Ростиславович	Худякин С.Р.
Булатов	Соломон	Казимирович	Булатов С.К.
Захаров	Марфа	Валерийович	Захаров М.В.

Рис. 9. Результат работы функции `show_initials`

2.4.2 Процедура

Для создания процедуры `age_to_type` необходимо осуществить проверки на то, что данный возрастной тип и вид абонеента существует. Если возрастного типа не существует, то он добавляется. Если не существует вида абонеента - он также добавляется с заданным ранее возрастным типом. Если возрастного типа нет, а вид абонеента есть, то сначала добавляется тип, а затем проводится проверка, предназначен ли для данного вида абонеента указанный возрастной тип. В случае, если такая запись не находится, она добавляется. Каждое действие логируется. Блок-схема процедуры представлена на картинке 10. Код процедуры приведен в листинге 20.

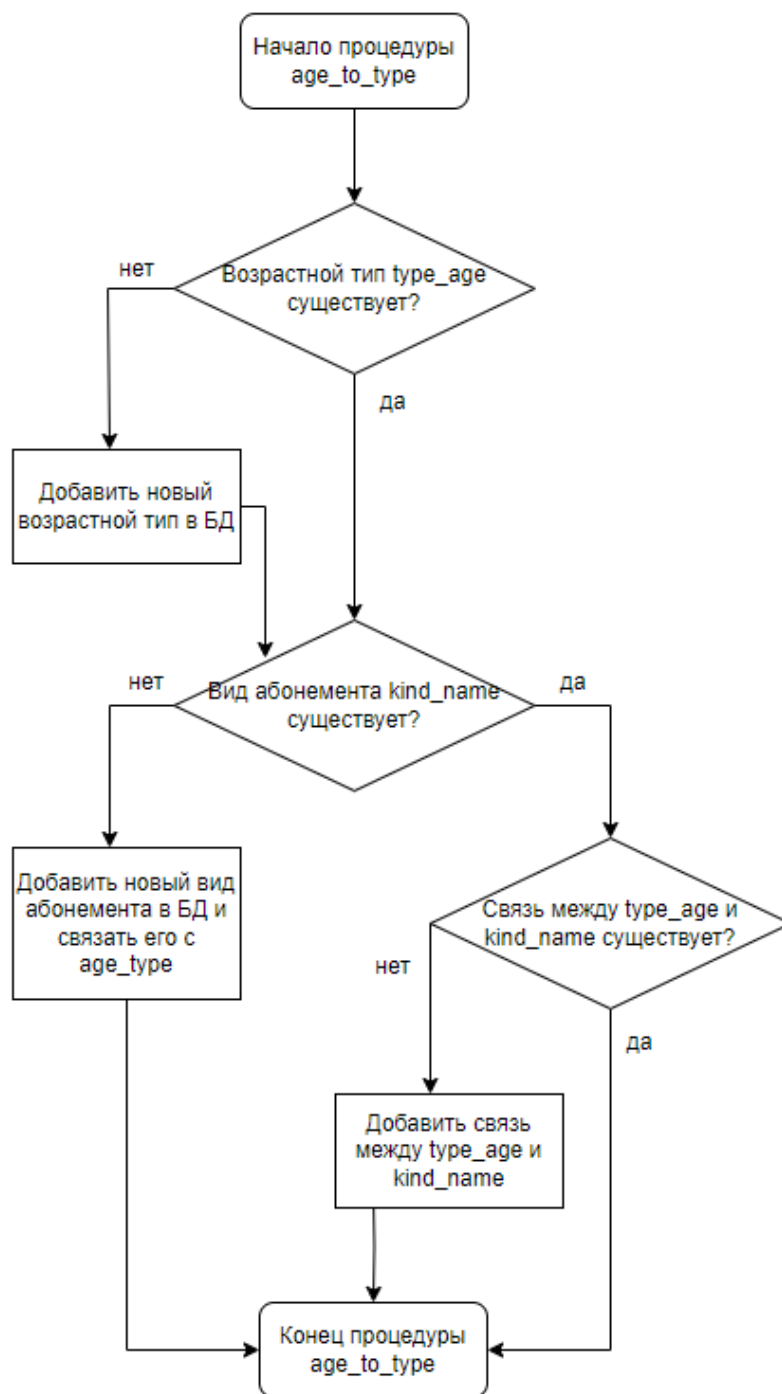


Рис. 10. Блок-схема процедуры

Процедура вызывается с помощью команды call. Пример вызова процедуры с новым возрастным типом и новым видом абонемента представлен на рисунках 11а - 11d.

	id_category	type_age
▶	1	детский
	2	взрослый
✱	NULL	NULL

(a) Список типов до использования процедуры

	id_kind_of_pass	kind_name	id_category
▶	1	обучение плаванию	1
	2	спортивное	1
	3	оздоровительное	1
	4	обучение плаванию	2
	5	аквааэробика	2
✱	NULL	NULL	NULL

(b) Список видов до использования процедуры

	id_category	type_age
▶	1	детский
	2	взрослый
	9	пенсион
✱	NULL	NULL

(c) Список типов после использования процедуры

	id_kind_of_pass	kind_name	id_category
▶	1	обучение плаванию	1
	2	спортивное	1
	3	оздоровительное	1
	4	обучение плаванию	2
	5	аквааэробика	2
	13	аквайога	9
✱	NULL	NULL	NULL

(d) Список видов после использования процедуры

Рис. 11. Изменения в базе данных при использовании процедуры

Тестовые примеры для процедуры, покрывающие все варианты реализации функции, приведены в таблице 4.

Таблица 4. Перечень тестовых примеров для процедуры

№	Описание проверяемого состояния								
1	Результат работы процедуры при добавлении нового возрастного тип и нового вида абонеента. Добавляются оба компонента, так как ранее в БД их не было. <table><tr><td></td><td>Лог</td></tr><tr><td>▶</td><td>Добавлен новый возрастной тип: "пенсион" с ID = 14</td></tr><tr><td>▶</td><td>Добавлен новый вид абонеента: "аквайога" с ID = 17</td></tr><tr><td>▶</td><td>Вид абонеента "аквайога" уже связан с возрастным типом "пенсион".</td></tr></table>		Лог	▶	Добавлен новый возрастной тип: "пенсион" с ID = 14	▶	Добавлен новый вид абонеента: "аквайога" с ID = 17	▶	Вид абонеента "аквайога" уже связан с возрастным типом "пенсион".
	Лог								
▶	Добавлен новый возрастной тип: "пенсион" с ID = 14								
▶	Добавлен новый вид абонеента: "аквайога" с ID = 17								
▶	Вид абонеента "аквайога" уже связан с возрастным типом "пенсион".								

№	Описание проверяемого состояния								
2	Результат работы при вызове процедуры с существующим возрастным типом и существующим видом абонента. Оба компонента уже есть в базе данных, поэтому повторно они не добавляются, также проверяется существование связи между ними. <table border="1"> <thead> <tr> <th></th><th>Лог</th></tr> </thead> <tbody> <tr> <td>▶</td><td>Найден существующий возрастной тип: "детский" с ID = 1</td></tr> <tr> <td>▶</td><td>Найден существующий вид абонента: "спортивное" с ID = 2</td></tr> <tr> <td>▶</td><td>Вид абонента "спортивное" уже связан с возрастным типом "детский".</td></tr> </tbody> </table>		Лог	▶	Найден существующий возрастной тип: "детский" с ID = 1	▶	Найден существующий вид абонента: "спортивное" с ID = 2	▶	Вид абонента "спортивное" уже связан с возрастным типом "детский".
	Лог								
▶	Найден существующий возрастной тип: "детский" с ID = 1								
▶	Найден существующий вид абонента: "спортивное" с ID = 2								
▶	Вид абонента "спортивное" уже связан с возрастным типом "детский".								
3	Результат работы при вызове процедуры с новым возрастным типом и существующим видом абонента. Новый возрастной тип добавляется в БД, существующий вид абонента - нет. Добавляется запись о новой связи между ними, так как ранее ее не было. <table border="1"> <thead> <tr> <th></th><th>Лог</th></tr> </thead> <tbody> <tr> <td>▶</td><td>Добавлен новый возрастной тип: "малыш" с ID = 15</td></tr> <tr> <td>▶</td><td>Найден существующий вид абонента: "оздоровительное" с ID = 3</td></tr> <tr> <td>▶</td><td>Добавлена новая запись: вид "оздоровительное", возрастной тип "малыш" (ID = 18).</td></tr> </tbody> </table>		Лог	▶	Добавлен новый возрастной тип: "малыш" с ID = 15	▶	Найден существующий вид абонента: "оздоровительное" с ID = 3	▶	Добавлена новая запись: вид "оздоровительное", возрастной тип "малыш" (ID = 18).
	Лог								
▶	Добавлен новый возрастной тип: "малыш" с ID = 15								
▶	Найден существующий вид абонента: "оздоровительное" с ID = 3								
▶	Добавлена новая запись: вид "оздоровительное", возрастной тип "малыш" (ID = 18).								

2.5 Транзакции

Постановка задачи.

1. Создать двух пользователей.
2. Выбрать уровень изоляции.
3. Выбрать аномалию и показать ее наличие или отсутствие на данном уровне изоляции.

Для выполнения задачи было создано два пользователя. Код создания пользователей представлен в листинге 10.

Пусть уровень изоляции будет Read Committed - чтение завершенных транзакций. Нужно проверить, есть ли на этом уровне аномалия неповторяющегося чтения, то есть совпадают ли результаты чтения данных на шагах 3 и 8 таблицы 5 при изменениях на шаге 6.

Листинг 10. Создание пользователей

```
1 create user 'user1'@'localhost' identified by 'pass1';
2 create user 'user2'@'localhost' identified by 'pass2';
3
4 grant all privileges on swimming_pool.* to 'user1'@'localhost';
5 grant all privileges on swimming_pool.* to 'user2'@'localhost';
6
```

Таблица 5. Проверка наличия аномалии

№	User1	User2
1	Установка уровня изоляции транзакции <pre>set session transaction isolation level read committed;</pre> <pre>Query OK, 0 rows affected (0.00 sec)</pre>	
2	Старт транзакции <pre>start transaction;</pre> <pre>Query OK, 0 rows affected (0.00 sec)</pre>	

№	User1	User2
3	Вывод данных о посетителе с id=555 <pre>select * from ' swimming_pool'. 'Visitor' where id_visitor = 555;</pre> <pre>***** 1. row ***** id_visitor: 555 visitor_surname: Богатырь visitor_name: Добрыня visitor_patronymic: Никитич visitor_date_of_birth: 2012-03-09 id_responsible: 2624 medical_certificate: 1 visitor_phone: 89737517894 1 row in set (0.01 sec)</pre>	
4		Установка уровня изоляции транзакции <pre>set session transaction isolation level read committed;</pre> <pre>Query OK, 0 rows affected (0.00 sec)</pre>
5		Старт транзакции <pre>start transaction;</pre> <pre>Query OK, 0 rows affected (0.00 sec)</pre>
6		Изменение данных транзакцией <pre>update ' swimming_pool'. 'Visitor' set visitor_name = "Алеша" , visitor_patronymic = " Попович" where id_visitor = 555;</pre> <pre>Query OK, 1 row affected (0.01 sec) Rows matched: 1 Changed: 1 Warnings: 0</pre>
7		Применение изменений <pre>commit;</pre> <pre>Query OK, 0 rows affected (0.00 sec)</pre>

№	User1	User2
8	Вывод данных о посетителе с id=555 <pre>select * from ' swimming_pool'.'Visitor' where id_visitor = 555;</pre> <pre>***** 1. row **** id_visitor: 555 visitor_surname: Богатырь visitor_name: Алеша visitor_patronymic: Попович visitor_date_of_birth: 2012-03-09 id_responsible: 2624 medical_certificate: 1 visitor_phone: 89737517894 1 row in set (0.00 sec)</pre>	

Из таблицы видно, что данные на шагах 3 и 8 первой транзакции не совпадают из-за изменения данных второй транзакцией на шаге 6. Очевидно, что аномалия неповторяющегося чтения присутствует на уровне изоляции Read Committed.

Заключение

Работа была выполнена в MySQL Workbench 8.0 CE и командной строке Windows.

Создано представление на основе ранее сформулированного запроса, проверена его обновляемость и написаны два запроса к данному представлению. Написаны триггеры на добавление, обновление и удаление для корректного подсчета статистики. Таблица со статистическими данными содержит количество абонементов конкретного типа. Созданы два пользователя с различными правами доступа, проверены их возможности. Создана функция, возвращающая фамилию и инициалы из полных фамилии, имени и отчества. Создана процедура, которая связывает возрастной тип с видом абонемента и при необходимости добавляет новые элементы. Если запись существует, повторная вставка данных в таблицу не проводится. Выбран уровень изоляции Read Committed и продемонстрировано наличие аномалии неповторяющегося чтения.

Работа была выполнена на ноутбуке с процессором AMD Ryzen 5 4600H, 8 ГБ оперативной памяти и операционной системой Windows 11.

Список литературы

- [1] Дейт К. «Введение в системы базы данных. 8-е изд.». М.: Вильямс, 2005. 1328 стр.
- [2] Кузнецов С.Д. «Базы данных. Модели и языки». М.: БИНОМ-ПРЕСС, 2008. 488 стр.
- [3] СУБД: для чего они нужны // habr : [сайт]. URL: <https://habr.com/ru/companies/amvera/articles/754702/> (дата обращения: 30.11.2025).

Приложение А «Исходный текст функции»

Листинг 19. Функция show_initials

```
1  delimiter //
2  create function show_initials (surname VARCHAR(40), name VARCHAR(15),
3    patronymic VARCHAR(20))
4    returns varchar(45)
5    deterministic
6    begin
7    declare res varchar(45);
8    set res = concat (surname, ' ', left(name, 1), '.'),
9    if (patronymic is not null, concat(left(patronymic, 1), '.'), '')
10   );
11   return res;
12 end //
delimiter ;
```

Приложение Б «Исходный текст процедуры»

Листинг 20. Процедура age_to_type

```
1  delimiter //
2  create procedure age_to_type(c_type_age varchar(8), k_kind_name
   varchar(17))
3  begin
4  declare c_age_id int;
5  declare k_kind_id int;
6  -- проверяем наличие возрастного типа
7  if exists (select 1 from category_of_pass where type_age = c_type_age)
   then
8  select id_category
9  into c_age_id
10 from category_of_pass
11 where type_age = c_type_age;
12 select concat ('Найден существующий возрастной тип: "', c_type_age, '" с ID
   = ', c_age_id) as 'Лог';
13 else
14 insert into category_of_pass (type_age) values (c_type_age);
15 set c_age_id = last_insert_id();
16 select concat ('Добавлен новый возрастной тип: "', c_type_age, '" с ID = ',
   c_age_id) as 'Лог';
17 end if;
18 -- проверяем наличие вида абонемента
19 if exists (select 1 from kind_of_pass where kind_name = k_kind_name)
   then
20 select id_kind_of_pass
21 into k_kind_id
22 from kind_of_pass
23 where kind_name = k_kind_name;
24 select concat ('Найден существующий вид абонемента: "', k_kind_name, '" с
   ID = ', k_kind_id) as 'Лог';
25 else
26 insert into kind_of_pass (kind_name, id_category) values (k_kind_name,
   c_age_id);
27 set k_kind_id = last_insert_id();
28 select concat ('Добавлен новый вид абонемента: "', k_kind_name, '" с ID =
   ', k_kind_id) as 'Лог';
29 end if;
30 -- проверяем наличие связи вида абонемента с возрастным типом
31 if exists (select 1 from kind_of_pass where kind_name = k_kind_name
   and id_category = c_age_id) then
32 select concat ('Вид абонемента "', k_kind_name, '" уже связан с возрастным
   типом "', c_type_age, '".') as 'Лог';
33 else
34 insert into kind_of_pass (kind_name, id_category) values (k_kind_name,
   c_age_id);
35 set k_kind_id = last_insert_id();
36 select concat ('Добавлена новая запись: вид "', k_kind_name, '",
   возрастной тип "', c_type_age, '" (ID = ', k_kind_id, ').') as 'Лог';
37 end if;
38 end //
39 delimiter ;
```